

Student LMS

AWS Deployment Automation Summary — Architecture Review

The deployment solution uses **AWS CDK with TypeScript** to define infrastructure, **PowerShell** to orchestrate deployment, and **AWS CloudFormation** to create and manage resources. It deploys eight Spring Boot microservices on ECS Fargate and serves the frontend globally through S3 and CloudFront.

1. End-to-End Deployment Flow

Validate prerequisites → Run tests → Create foundation resources
→ Build and push eight Docker images → Deploy Fargate and API layer
→ Deploy CloudFront and WAF → Upload frontend → Display endpoints

2. Deployment Stack Structure

Stack	Region	Responsibility
foundation	Application region	VPC, RDS PostgreSQL, Redis, DynamoDB, S3, SNS/SQS, secrets and ECR.
application	Application region	ECS Fargate, internal ALB, target groups, API Gateway, scaling and logs.
edge	us-east-1	CloudFront, WAF, frontend S3, optional ACM certificate and Route 53.

3. Files and Responsibilities

File	Deployment responsibility
<code>scripts/aws/deploy.ps1</code>	Main end-to-end deployment orchestrator.
<code>scripts/aws/common.ps1</code>	Shared command validation, AWS account lookup, error handling and stack-output functions.
<code>scripts/aws/deploy-frontend.ps1</code>	Uploads frontend assets to S3 and invalidates CloudFront.
<code>scripts/aws/status.ps1</code>	Shows CloudFormation status and ECS desired/running task counts.
<code>scripts/aws/destroy.ps1</code>	Removes non-production stacks safely in reverse order.
<code>infrastructure/bin/lms.ts</code>	CDK entry point; reads environment settings and creates all three stacks.
<code>infrastructure/lib/foundation-stack.ts</code>	Creates networking, databases, storage, messaging and container repositories.
<code>infrastructure/lib/application-stack.ts</code>	Creates the Fargate runtime, ALB routing, API Gateway and observability resources.
<code>infrastructure/lib/edge-stack.ts</code>	Creates global frontend delivery, WAF protection and optional DNS/TLS.
<code>infrastructure/lib/config.ts</code>	Centralizes service names, ports, routes and ALB rule priorities.
<code>*-service/Dockerfile</code>	Builds each Spring Boot service into a non-root deployable container.

4. Foundation Stack

`foundation-stack.ts` creates resources required before containers can start:

- VPC spanning two Availability Zones, with public, private application and isolated data subnets.
- RDS PostgreSQL 15, generated credentials, encryption, backups and production Multi-AZ support.
- ElastiCache Redis with authentication and encryption in transit and at rest.
- Private S3 buckets for course content, submissions and certificates.
- DynamoDB activity and audit tables with on-demand capacity, TTL and recovery support.
- SNS domain-event topic, SQS work queue and dead-letter queue.
- Eight private ECR repositories with immutable tags and image scanning.

5. Container Build and Publication

Each service Dockerfile performs a multi-stage build:

1. Build the service and shared `common` module with Maven.
2. Package the executable Spring Boot JAR.
3. Create a smaller runtime image running as a non-root user.
4. Expose the service port and configure an Actuator health check.

The deployment script authenticates Docker with ECR, builds all eight images, tags them using the Git commit or supplied release tag, and pushes them to their dedicated repositories.

6. Application Stack

`application-stack.ts` creates the backend execution layer:

- One ECS cluster and one Fargate service per microservice.
- Private task networking with no public IP addresses.
- Secrets Manager injection for database, JWT and Redis credentials.
- CloudWatch log groups, Container Insights and service CPU alarms.
- CPU/memory auto-scaling, deployment circuit breaker and automatic rollback.
- An internal ALB with an IP target group and health check for each service.
- API Gateway HTTP API with a VPC Link to the private ALB.

Request flow

Client → CloudFront/WAF → API Gateway → VPC Link
→ Internal Application Load Balancer → ECS Fargate service

Example path routing

Path	Service	Port
<code>/api/v1/auth*</code>	Auth	8081
<code>/api/v1/users*</code>	User	8082
<code>/api/v1/courses*</code>	Course	8083
<code>/api/v1/enrollments*</code>	Enrolment	8084
<code>/api/v1/assessments*</code>	Assessment	8085
<code>/api/v1/progress*</code>	Progress	8086
<code>/api/v1/students/*/preferences*</code>	Notification	8087
<code>/api/v1/admin/logs*</code>	Logging	8088

7. Edge Stack

`edge-stack.ts` is deployed in `us-east-1`, which is required for CloudFront-scoped WAF and CloudFront ACM certificates.

- Private S3 bucket for the frontend.
- CloudFront distribution with S3 Origin Access Control.
- `/api/*` behavior routed to the regional API Gateway.
- HTTPS enforcement, compression, cache policies and SPA error fallback.
- AWS managed WAF rules for common attacks, bad inputs and IP reputation.
- Rate limiting, with Shield Standard protection supplied automatically.
- Optional ACM certificate and Route 53 alias for a custom domain.

8. Main Script Execution

`deploy.ps1` executes the following sequence:

1. Validate AWS CLI, Docker, Node, npm and AWS credentials.
2. Install and compile the pinned CDK dependencies.
3. Bootstrap CDK in the application region and `us-east-1`.
4. Run Maven tests and quality gates.
5. Deploy the foundation stack.
6. Build and push all eight immutable Docker images.
7. Deploy the application stack.
8. Read the API Gateway endpoint from CloudFormation outputs.
9. Deploy the edge stack.
10. Publish the frontend and invalidate CloudFront.
11. Print the application and API URLs.

9. Environment Controls

Capability	Development	Production
Fargate tasks	1 per service	At least 2 per service
NAT gateways	1	2
RDS	Single-AZ	Multi-AZ
Redis	Single node	Replica and automatic failover
Resource deletion	Permitted	Retention and termination protection

10. Current Application Gaps

The deployment infrastructure is complete, but production readiness also requires replacing the mock S3 presigner, implementing SES/WebSocket notification adapters, adding full OpenTelemetry instrumentation, and restricting API CORS to the final production domain.

Architecture Review Statement

The solution uses AWS CDK and PowerShell to provision a multi-AZ, private Fargate microservice platform in three controlled CloudFormation stacks: foundation resources, application/API resources, and global CloudFront/WAF edge resources. It automates testing, Docker publication, deployment, observability, frontend delivery and environment-safe cleanup.